

Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An Autonomous Institution Affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	VI
Subject Code & Name	UCS2612 – Machine Learning Algorithms Laboratory		
Academic Year	2025–2026 (Even)	Batch	2023–2027
Name	Madussree Ravi	Register No.	3122235001074

Experiment 5: Perceptron vs Multilayer Perceptron with Hyperparameter Tuning

Objective

To implement and compare the performance of a Single-Layer Perceptron (Model A) and a Multilayer Perceptron (Model B) on the English Handwritten Characters dataset. The experiment involves implementing Model A from scratch and tuning Model B using various hyperparameters.

Dataset Description

The **English Handwritten Characters Dataset** consists of grayscale images representing 62 classes (0–9, A–Z, and a–z).

- **Total Samples:** 3,410.
- **Samples per Class:** 55 images.
- **Preprocessing Steps:**
 - Images were resized to 40×30 pixels.
 - Flattened into a 1D vector of 1,200 features.
 - Normalized by scaling pixel values to the range $[0, 1]$.
 - Labels were encoded using `LabelEncoder`.

Implementation Details

Model A: Single-Layer Perceptron (PLA)

The Perceptron was implemented from scratch using a step activation function. To handle the 62-class problem, a **One-vs-Rest (OvR)** strategy was employed, where one perceptron was trained for each class.

- **Activation:** Step Function.
- **Learning Rate:** 0.01.
- **Epochs:** 5.

Model B: Multilayer Perceptron (MLP)

The MLP was implemented using `MLPClassifier` from `sklearn`. Three different configurations were tested during the hyperparameter tuning phase.

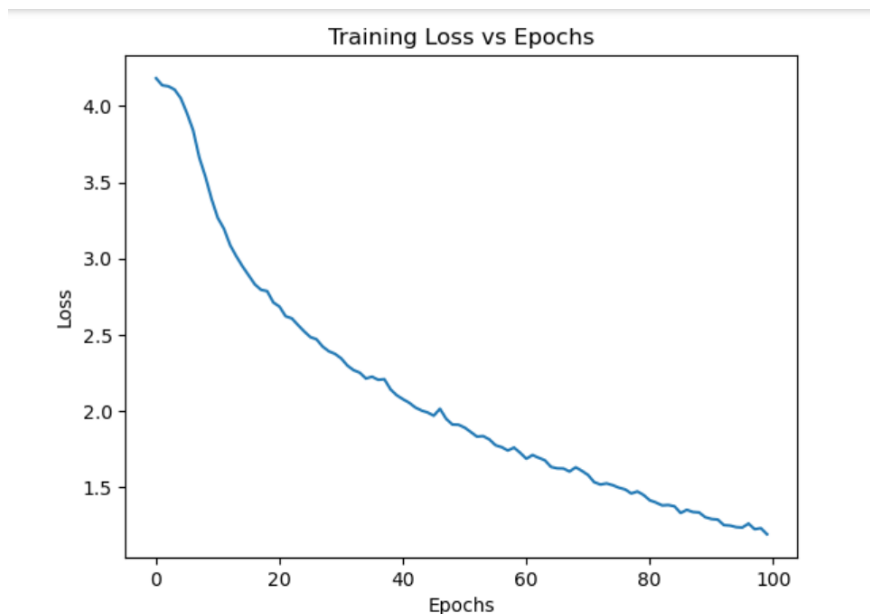


Figure 1: Training Loss vs. Epochs for the Tuned MLP (Model 2)

Performance Evaluation

Table 1: Hyperparameter Tuning Results for MLP

Hidden Layers	Activation	Optimizer	Learning Rate	Batch Size	Accuracy (%)
1 (128)	ReLU	SGD	0.01	32	36.80%
2 (256–128)	ReLU	Adam	0.001	64	41.35%
3 (512–256–128)	Tanh	Adam	0.0005	64	17.30%

Table 2: Overall Performance Comparison (Tuned Models)

Model	Accuracy (%)	Precision	Recall	F1-score
PLA (OvR)	1.46%	0.02	0.01	0.01
MLP (Tuned)	41.35%	0.44	0.41	0.41

<matplotlib.image.AxesImage at 0x1ce0f7f6e40>

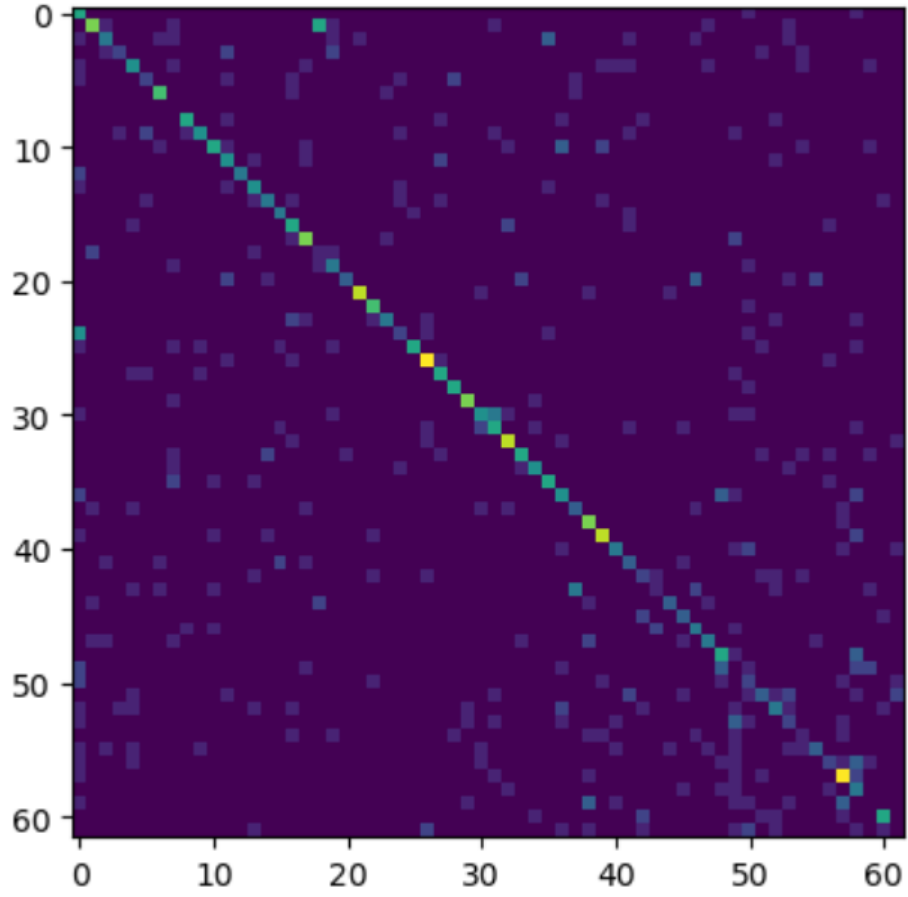


Figure 2: Confusion Matrix for Tuned MLP (Model 2) on 62 Classes

Table 3: Training Convergence Comparison

Model	Epochs	Final Training Loss	Convergence Behavior
PLA	5	N/A	Non-convergent
MLP (Model 2)	100	0.7481	Stable (Warning: Max Iter)

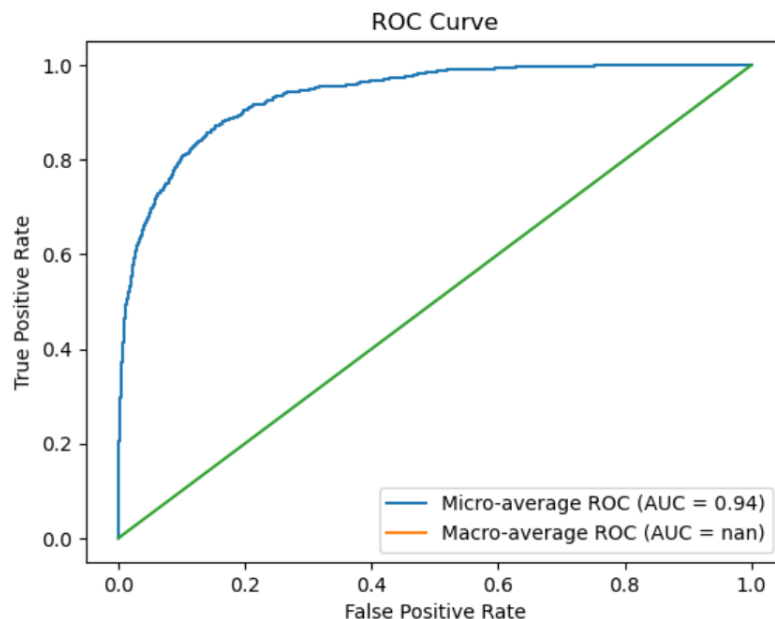


Figure 3: Micro and Macro-average ROC Curves for Tuned MLP

Observation Questions

1. **Why does PLA underperform compared to MLP?** PLA is a linear classifier that relies on the data being linearly separable. Handwritten characters contain complex, non-linear patterns that a single-layer perceptron cannot capture. MLP, with its hidden layers and non-linear activation functions (ReLU), can learn these complex decision boundaries.
2. **Which hyperparameter had the most impact on MLP performance?** The **hidden layer architecture** and the **optimizer** had the most significant impact. Moving from 1 to 2 hidden layers improved accuracy, but the transition to a very deep 3-layer architecture with Tanh activation caused a significant drop to 17.30%.
3. **How does optimizer choice affect convergence?** In this experiment, the **Adam** optimizer generally provided better results and faster progress than **SGD** for this specific dataset size, though Model 2 still triggered a convergence warning at 100 iterations.
4. **Does increasing hidden layers always improve performance?** No. As seen in Model 3 (512-256-128), increasing the depth without appropriate adjustments to the activation function or learning rate led to poor performance (17.30%) compared to the 2-layer model (41.35%).
5. **How can overfitting be detected and mitigated?** Overfitting is detected when training accuracy is much higher than test accuracy. It can be mitigated using **regularization** (alpha), **dropout**, or **early stopping**.

Conclusion

The experiment successfully compared PLA and MLP. The results demonstrate that while PLA is suitable for simple linear tasks, deep learning models like MLP are essential for complex image

recognition. Hyperparameter tuning revealed that a 2-layer ReLU network with the Adam optimizer was the most effective configuration, achieving an accuracy of 41.35%.

Learning Outcomes

- Implemented the Perceptron Learning Algorithm from scratch.
- Applied the One-vs-Rest strategy for multi-class classification.
- Tuned MLP hyperparameters including layers, activation functions, and optimizers.
- Evaluated model performance using Accuracy, Precision, Recall, and F1-score.